

08 - Algorithms & Problem Solving

Level4

Problem #01: Number To Text

التعريف: 🔍

المطلوب هو كتابة برنامج بلغة C++ يقوم بقراءة رقم من المستخدم ثم طباعته على شكل نص (كلمات إنجليزية تمثل الرقم)

مثال: 💡

Enter a Number? 5843

Five Thousands Eight Hundreds Forty Three

الشرح: 📖

يعتمد البرنامج على الدوال (Functions) لتقسيم المهام:

- `ReadNumber()` لقراءة الرقم من المستخدم
- `NumberToText()` لتحويل الرقم إلى نص بحسب قيمته
- `main()` لاستدعاء الدوال وتنفيذ البرنامج

الإجراءات: 📁

1 قراءة الرقم:

```
int ReadNumber()
{
    int number;
    cout << "\nEnter a Number? ";
    cin >> number;
    return number;
}
```

2 تحويل الرقم إلى نص:

الدالة `NumberToText(int number)` تحتوي على سلسلة من الشروط للتحقق من قيمة الرقم وتحويله إلى كلمات إنجليزية

أهم الحالات التي تتعامل معها: 📖

الوحدات (0-9):

يتم استخدام مصفوفة `units` لتحويل الأرقام إلى كلمات مثل "Zero", "One", "Two", ...

الأرقام بين (10-19):

يتم التعامل معها كمجموعة خاصة باستخدام مصفوفة `teens` لأن أسماءها لا تتبع قاعدة الجمع العادي

العشرات (20-99):

تستخدم مصفوفة `tens` وتُضاف إليها الوحدات المتبقية

مثال: 43 → Forty Three 💡

المئات (100-999):

يتم أولاً استخراج الجزء الخاص بالمئات ثم الباقي من خلال %

مثال: 584 → **Five Hundreds Eighty Four** 💡

الآلاف والملايين والمليارات: 🟡

يتم التعامل معها بشكل مشابه، بتقسيم الرقم حسب القيم (ألف، مليون، مليار) واستدعاء الدالة نفسها بشكل متكرر

(Recursion)

مثال: 5843 → **Five Thousands Eight Hundreds Forty Three** 💡

ملاحظة: 📌

البرنامج يعتمد على الاستدعاء الذاتي (Recursion) داخل الدالة **NumberToText()** لتحليل الرقم إلى أجزاء (آلاف، مئات، عشرات، وحدات)

يجب الانتباه إلى كتابة الكلمات بصيغة الجمع الصحيحة (مثل "Hundred" وليس "Hundreds") لتحسين الإخراج اللغوي 🟡

تفسير: 🤖

عند إدخال رقم مثل 5843، الدالة تقوم بتحليله كالتالي: 🟡

"Five Thousands" → $5 = 1000 \div 5843$ 🟢

"Eight Hundreds Forty Three" → 843 🟢

ثم تجمع النتيجة لتنتج النص النهائي

خلاصة: ✨

البرنامج يقرأ رقمًا صحيحًا من المستخدم ويحوّله إلى صيغة نصية باللغة الإنجليزية ✓

يعتمد على التقسيم الشرطي والتكرار الذاتي لمعالجة الأرقام من 0 إلى المليارات ✓

مثال نموذجي لتطبيق recursion و string arrays في معالجة الأرقام النصية ✓

Problem #02: Leap Year

التعريف:

- المطلوب هو كتابة برنامج بلغة C++ يتحقق مما إذا كانت السنة المدخلة سنة كبيسة (Leap Year) أم لا
- السنة الكبيسة هي التي تحتوي على 366 يوماً بدلاً من 365، ويكون شهر فبراير فيها 29 يوماً

الشرح:

- يعتمد البرنامج على القاعدة الرياضية التالية لتحديد السنة الكبيسة:
 - السنة تكون كبيسة إذا كانت:
 - تقسم تماماً على 4 أي $year \% 4 == 0$
 - باستثناء السنوات التي تنتهي بـ "00" (قرنية):
 - فهي لا تعتبر كبيسة إلا إذا كانت تقسم تماماً على 400 أي $year \% 400 == 0$
- أي أن:
 - 2000 كبيسة لأنها تقسم على 400
 - 1900 ليست كبيسة لأنها لا تقسم على 400 رغم أنها تقسم على 100
 - 1996 كبيسة لأنها تقسم على 4 فقط

الإجراءات:

1 قراءة السنة من المستخدم:

- يتم ذلك باستخدام الدالة `readYear()` التي تطلب إدخال السنة وتعيدها كعدد صحيح

```
int readYear()
{
    int year;
    cout << "Please enter a year to check? ";
    cin >> year;
    return year;
}
```

2 التحقق من كون السنة كبيسة:

- يتم ذلك عبر الدالة `isLeapYear(int year)` التي تطبق القواعد السابقة بالترتيب:
 - إن كانت السنة تقسم على 400 كبيسة
 - وإلا إن كانت تقسم على 100 غير كبيسة
 - وإلا إن كانت تقسم على 4 كبيسة
 - وإلا فهي غير كبيسة

```
bool isLeapYear(int year)
{
    if (year \% 400 == 0)
        return true;
    else
        if (year \% 100 == 0)
            return false;
        else if (year \% 4 == 0)
            return true;
        else
            return false;
}
```

3 عرض النتيجة للمستخدم:

في الدالة **main()** يتم استدعاء الدالتين السابقتين ثم طباعة النتيجة المناسبة:

```
if (isLeapYear(year))
    cout << year << " is a leap year.";
else
    cout << year << " is not a leap year.";
```

مثال: 💡

السنة	النتيجة
1996	Leap Year
1900	Not Leap Year
2000	Leap Year
2004	Leap Year
2010	Not Leap Year

ملاحظة: 📌

يتم التحقق من القسمة باستخدام معامل الباقي %
ترتيب الشروط في الدالة مهم جدًا لأن القاعدة الخاصة بالـ 400 يجب أن تُفحص أولاً

تفسير: 🤖

السنة التي تقسم على 100 ليست دائمًا كبيسة لأن التقويم الميلادي يضيف يومًا كل 4 سنوات فقط، لكن لإصلاح انحراف بسيط في الحساب، تم استثناء القرون غير القابلة للقسمة على 400 من كونها كبيسة

✨ خلاصة:

- ✓ السنة كبيسة إذا كانت تقسم على 4 ولا تقسم على 100، أو كانت تقسم على 400
- ✓ البرنامج يقرأ السنة من المستخدم ويطبع النتيجة مباشرة
- ✓ من الأمثلة المشهورة:
 - 2000 كبيسة
 - 1900 ليست كبيسة
 - 2024 كبيسة

Problem #03: Leap Year (One Line of Code)

التعريف:

الهدف من هذا البرنامج هو التحقق مما إذا كانت السنة المدخلة كبيسة أم لا، ولكن باستخدام سطر واحد فقط من الكود داخل دالة التحقق

يتم ذلك عبر استعمال العوامل المنطقية (Logical Operators) بدلاً من التفرعات الشرطية المتعددة **if / else**

الشرح:

القاعدة الأساسية لتحديد السنة الكبيسة هي نفسها كما في المسألة السابقة:

السنة كبيسة إذا كانت:

تقسم على 4

ولا تقسم على 100

إلا إذا كانت تقسم على 400

الفرق في هذا التمرين هو اختصار القواعد السابقة كلها في شرط منطقي واحد باستخدام معاملات **AND** و **OR**

الإجراءات:

1) قراءة السنة من المستخدم:

باستخدام الدالة **readYear()** التي تطلب من المستخدم إدخال السنة وتعيدها كعدد صحيح

```
int readYear()
{
    int year;
    cout << "Please enter a year to check? ";
    cin >> year;
    return year;
}
```

2) التحقق من كون السنة كبيسة (بسطر واحد فقط):

في الدالة **isLeapYear()** يتم استخدام تعبير منطقي واحد فقط:

```
return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
```

شرح الشرط:

(year % 4 == 0 && year % 100 != 0) السنة تقسم على 4 ولكن لا تقسم على 100

(year % 400 == 0) أو السنة تقسم على 400 (حالة استثناء للقرون)

النتيجة النهائية هي **true** إذا تحقق أي من الشرطين

3) عرض النتيجة:

في **main()** يتم استدعاء الدالة وطباعة النتيجة:

```
if (isLeapYear(year))
    cout << "\nYes, Year [" << year << "] is a leap year.\n";
else
    cout << "\nNo, Year [" << year << "] is NOT a leap year.\n";
```

مثال:

السنة	النتيجة
1996	Leap Year

Not Leap Year	1900
Leap Year	2000
Leap Year	2004
Not Leap Year	2010

ملاحظة:

تم استخدام العاملين المنطقيين:

&& (AND) لربط شرطين يجب أن يتحققا معًا

|| (OR) لربط شرطين يكفي تحقق أحدهما ليكون الناتج صحيحًا

تفسير:

هذا الحل أكثر اختصارًا وكفاءة لأن المترجم سيقم الشرط المنطقي في سطر واحد دون الحاجة لتداخل عدة **if** متتالية رغم اختصاره، إلا أنه يحقق نفس منطق المسألة السابقة بدقة تامة

خلاصة:

✓ تم تبسيط منطق السنة الكبيسة إلى شرط منطقي واحد باستخدام **&&** و **||**

✓ الشرط يتحقق فقط عندما:

السنة تقسم على 4 ولا تقسم على 100، أو تقسم على 400

■ Problem #04: Number of Days–Hours–Minutes–Seconds in a Year

التعريف: 🔍

الهدف من البرنامج هو حساب عدد الأيام، الساعات، الدقائق، والثواني في سنة معينة، مع الأخذ بعين الاعتبار ما إذا كانت السنة كبيسة أم لا

الشرح: 📖

السنة الكبيسة تحتوي على 366 يومًا، بينما السنة العادية تحتوي على 365 يومًا
يتم حساب بقية القيم (الساعات، الدقائق، الثواني) بالاعتماد على عدد الأيام في السنة
العلاقة الحسابية بين الوحدات هي:
1 يوم = 24 ساعة
1 ساعة = 60 دقيقة
1 دقيقة = 60 ثانية

الإجراءات: 🏠

1) قراءة السنة من المستخدم:

باستخدام الدالة `readYear()` التي تطلب من المستخدم إدخال السنة وتعيدها كعدد صحيح

```
int readYear()
{
    int year;
    cout << "Please enter a year to check? ";
    cin >> year;
    return year;
}
```

2) التحقق من كون السنة كبيسة:

باستخدام الدالة `isLeapYear(int year)` التي تعتمد على القاعدة الرياضية التالية:

```
return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
```

أي أن السنة كبيسة إذا كانت:

تقسم على 4

ولا تقسم على 100

أو تقسم على 400

3) حساب عدد الأيام والساعات والدقائق والثواني:

يتم في الدالة `printYearDetails(int year)` كما يلي:

```
int days = isLeapYear(year) ? 366 : 365;
int hours = days * 24;
int minutes = hours * 60;
int seconds = minutes * 60;
```

ثم يتم عرض النتائج على الشاشة باستخدام `cout`.

4) البرنامج الرئيسي:

في `main()` يتم استدعاء الدالتين:

```
int year = readYear();
printYearDetails(year);
```

مثال: 💡

Please enter a year to check? 2000

Number of Days in Year[2000] is 366
Number of Hours in Year [2000] is 8784
Number of Minutes in Year [2000] is 527040
Number of Seconds in Year [2000] is 31622400

Please enter a year to check? 1971

Number of Days in Year [1971] is 365
Number of Hours in Year [1971] is 8760
Number of Minutes in Year [1971] is 525600
Number of Seconds in Year [1971] is 31536000

ملاحظة: 📌

تم استخدام المعامل الثلاثي (Ternary Operator) ؟ : لتحديد عدد الأيام بسرعة بناءً على نتيجة الدالة `isLeapYear()`

هذه الطريقة تجعل الكود أكثر اختصارًا ووضوحًا من استخدام جمل شرطية طويلة

تفسير: 🤖

إذا كانت السنة كبيسة، تضاف 24 ساعة إضافية (أي يوم واحد) إلى مجموع الساعات في السنة، مما ينعكس أيضًا على عدد الدقائق والثواني
الحسابات متسلسلة: كل ناتج يعتمد على الذي قبله لضمان الدقة في التحويل بين الوحدات

خلاصة: ✨

السنة الكبيسة = 366 يومًا → 8784 ساعة → 527040 دقيقة → 31622400 ثانية ✓
السنة العادية = 365 يومًا → 8760 ساعة → 525600 دقيقة → 31536000 ثانية ✓
البرنامج يستخدم دوال منظمة لحساب القيم بدقة ووضوح ✓

Problem #05: Number Of Days, Hours, Minutes, Seconds In a Month

التعريف:

الهدف برنامج لحساب عدد الأيام والساعات والدقائق والثواني في شهر معين من سنة معينة يحددها المستخدم

الشرح:

البرنامج يطلب من المستخدم:

1 إدخال السنة

2 إدخال رقم الشهر

ثم يقوم بحساب عدد الأيام في هذا الشهر (مع الأخذ بالاعتبار ما إذا كانت السنة كبيسة أم لا)، ومن ثم يحسب:

عدد الساعات = الأيام $\times 24$

عدد الدقائق = الساعات $\times 60$

عدد الثواني = الدقائق $\times 60$

الإجراءات:

1 دالة `readYear()`

تطلب من المستخدم إدخال السنة وتعيدها كقيمة

2 دالة `isLeapYear(int year)`

تتحقق إذا كانت السنة كبيسة أم لا

السنة الكبيسة: تقبل القسمة على 4 ولا تقبل على 100، إلا إذا كانت تقبل القسمة على 400

3 دالة `readMonth()`

تطلب من المستخدم إدخال رقم الشهر

4 دالة `getDaysInMonth(int month, int year)`

تحدد عدد الأيام في الشهر المحدد

الشهور 1,3,5,7,8,10,12 → 31 يوم

الشهور 4,6,9,11 → 30 يوم

الشهر 2 → 28 أو 29 يوم حسب السنة الكبيسة

5 دالة `printMonthDetails()`

تحسب الأيام والساعات والدقائق والثواني وتعرضها للمستخدم

6 دالة `main()`

تنفذ جميع الخطوات السابقة بالتسلسل.

مثال:

Please enter a year to check? 2000

Please enter a Month to check? 2

Number of Days in Month [2] is 29

Number of Hours in Month [2] is 696

Number of Minutes in Month [2] is 41760

Number of Seconds in Month [2] is 2505600

ملاحظة:

إذا أدخل المستخدم رقم شهر غير صالح (أكبر من 12 أو أصغر من 1)، فستعيد الدالة القيمة 0

✨ خلاصة:

✓ البرنامج يطبق مفاهيم:

الدوال

الشروط

العمليات الحسابية

استخدام switch-case

Problem #06: Number of Days In a Month (Short Logic)

التعريف: 🔍

الهدف برنامج لحساب عدد الأيام في شهر محدد من سنة معينة، باستخدام منطق مختصر (سطر واحد لتحديد عدد الأيام)

الشرح: 📖

البرنامج يعتمد على منطق بسيط ومباشر بدلاً من استخدام عدة شروط يتم تخزين عدد الأيام لكل شهر في مصفوفة، ويُستخدم الشرط الثلاثي لتحديد ما إذا كان الشهر هو فبراير (2) ومعرفة إن كانت السنة كبيسة

الإجراءات: 📌

1 دالة ReadYear()

تطلب من المستخدم إدخال السنة وتُعيدها

2 دالة ReadMonth()

تطلب من المستخدم إدخال رقم الشهر وتُعيده

3 دالة IsLeapYear(int year)

تتحقق إذا كانت السنة كبيسة

كبيسة إذا كانت:

(تقسم على 4 ولا تقسم على 100) أو (تقسم على 400)

4 دالة NumberOfDaysInAMonth(int month, int year)

تستخدم مصفوفة:

```
int NumberOfDays[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

ثم تُعيد القيمة بناءً على الشرط الثلاثي:

```
return (month == 2) ? (IsLeapYear(year) ? 29 : 28) : NumberOfDays[month - 1];
```

5 دالة main()

تنفذ قراءة السنة والشهر وتعرض عدد الأيام مباشرة

مثال: 💡

Please enter a year to check? 2000

Please enter a Month to check? 2

Number of Days in Month [2] is 29

ملاحظة: 📌

إذا كان رقم الشهر أقل من 1 أو أكبر من 12، تُعاد القيمة 0

خلاصة: ✨

البرنامج يعتمد على: ✓

المصفوفات **Arrays**

الشروط الثلاثية (Ternary Operator)

الدوال **Functions**

Problem #07: Day Name

التعريف: 🔍

الهدف برنامج يقوم بقراءة تاريخ (يوم، شهر، سنة) من المستخدم ثم يعرض:
ترتيب اليوم في الأسبوع (من 0 إلى 6)
واسم اليوم مثل (Sun, Mon, Tue) ...

الشرح: 📖

البرنامج يعتمد على حساب ترتيب اليوم داخل الأسبوع باستخدام معادلة رياضية مبنية على تقويم **Gregorian**، ثم تحويل هذا الترتيب إلى اسم اليوم

الإجراءات: 📦

1 دالة ReadYear()

تطلب من المستخدم إدخال السنة وتعيدها

2 دالة ReadMonth()

قراءة رقم الشهر من المستخدم

3 دالة ReadDay()

قراءة رقم اليوم من المستخدم

4 دالة IsLeapYear(int year)

تتحقق إن كانت السنة كبيسة:

كبيسة إذا كانت:

$(year \% 4 == 0 \ \&\& \ year \% 100 != 0) \ || \ (year \% 400 == 0)$

5 دالة NumberOfDaysInAMonth(int month, int year)

تُرجع عدد الأيام في الشهر حسب السنة (29 في شباط إذا كبيسة)

6 دالة DayOfWeekOrder(int day, int month, int year)

معادلة حساب ترتيب اليوم في الأسبوع:

$(day + y + (y / 4) - (y / 100) + (y / 400) + ((31 * m) / 12)) \% 7;$

حيث:

$a = (14 - month) / 12$

$y = year - a$

$m = month + (12 * a) - 2$

→ الناتج هو رقم اليوم:

0 = Sun, 1 = Mon, 2 = Tue, ...

7 دالة DayShortName(int dayOrder)

تُرجع اسم اليوم باستخدام مصفوفة:

`string arrDayNames[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};`

8 دالة main()

يقرأ القيم، يحسب الترتيب، ثم يطبع التاريخ واسم اليوم

مثال: 💡

Please enter a year to check? 2022

Please enter a Month to check? 9

Please enter a Day? 20

Date: 20/9/2022

Day Order : 2
Day Name: Tue

ملاحظة: 

إذا كان إدخال الشهر أو اليوم غير صحيح، فقد تكون النتيجة غير دقيقة لأن البرنامج لا يتحقق من صلاحية التاريخ 

خلاصة: 

البرنامج يستخدم: 

العمليات الحسابية لتحديد اليوم 

المعادلات التقويمية (Gregorian formula) 

المصفوفات لتخزين أسماء الأيام 

Problem #08: Month Calendar

التعريف:

الهدف برنامج يقوم بطباعة تقويم شهر معين من سنة محددة، بحيث يعرض الأيام بشكل منسق بدءًا من اليوم الصحيح للأسبوع (Saturday إلى Sunday)

الشرح:

البرنامج يعتمد على تحديد اليوم الذي يبدأ به الشهر (مثلًا: خميس أو أحد)، ثم طباعة الأيام بالترتيب مع الفراغات المناسبة قبل اليوم الأول.

الإجراءات:

1 دالة ReadYear()

تطلب من المستخدم إدخال السنة

2 دالة ReadMonth()

قراءة رقم الشهر من المستخدم (من 1 إلى 12)

3 دالة IsLeapYear(int year)

التحقق إذا كانت السنة كبيسة:

السنة كبيسة إذا كانت:

$(year \% 4 == 0 \ \&\& \ year \% 100 != 0) \ || \ (year \% 400 == 0)$

4 دالة NumberOfDaysInAMonth(int month, int year)

تُعيد عدد الأيام في الشهر المحدد، مع الأخذ بالحسبان إن كان شباط 28 أو 29 يومًا.

5 دالة DayOfWeekOrder(int day, int month, int year)

تُحسب ترتيب اليوم من الأسبوع باستخدام الصيغة:

$(day + y + (y / 4) - (y / 100) + (y / 400) + ((31 * m) / 12)) \% 7;$

النتائج:

0 = Sun, 1 = Mon, 2 = Tue, ... إلخ.

6 دالة DayShortName(int dayOrder)

تُعيد اسم اليوم المختصر بناءً على الترتيب:

"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"

7 دالة MonthShortName(int monthNumber)

تُعيد اسم الشهر المختصر مثل:

"Jan", "Feb", "Mar", ..., "Dec"

8 دالة PrintMonthCalendar(int month, int year)

تعرض التقويم بالشكل التالي:

حساب أول يوم في الشهر

طباعة اسم الشهر والعناوين (...Sun, Mon, Tue)

ترك فراغات قبل بداية الشهر

طباعة الأيام بترتيبها الصحيح

مثال:

Please enter a year to check? 2022

Please enter a Month to check? 9

_____Sep
Sun Mon Tue Wed Thu Fri Sat

				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

ملاحظة: 📌

- يبدأ العد من الأحد كالיום الأول في الأسبوع
- التنسويق يعتمد على دالة **printf** للمحاذاة الدقيقة
- لا يوجد تحقق من صحة رقم الشهر، لذا إدخال رقم خارج 1-12 سيعيد 0 أيام

تفسير: 🤖

- المعادلة المستخدمة في **DayOfWeekOrder** هي خوارزمية زيلر (Zeller's Congruence) المعدلة، وهي طريقة دقيقة لتحديد اليوم لأي تاريخ ميلادي

خلاصة: ✨

- البرنامج يستخدم مفاهيم: ✓
- دوال متعددة (modular functions)
- حساب اليوم رياضياً
- الطباعة المنسقة باستخدام **printf**

Problem #09: Year Calendar

التعريف:

الهدف برنامج يقوم بطباعة التقويم الكامل لسنة معينة، بحيث يعرض جميع الأشهر من يناير إلى ديسمبر مع الأيام بترتيبها الصحيح

الشرح:

البرنامج يعتمد على نفس منطق التقويم الشهري (في Problem #08)، لكن هنا يتم استدعاء الدالة الخاصة بطباعة كل شهر ضمن حلقة تمر على الأشهر الـ 12 للسنة

الإجراءات:

1 دالة ReadYear()

تطلب من المستخدم إدخال السنة

2 دالة IsLeapYear(int year)

تحديد ما إذا كانت السنة كبيسة:

السنة كبيسة إذا كانت:

$(year \% 4 == 0 \ \&\& \ year \% 100 != 0) \ || \ (year \% 400 == 0)$

3 دالة NumberOfDaysInAMonth(int month, int year)

تعيد عدد الأيام في كل شهر (مع تعديل فبراير إذا كانت السنة كبيسة)

4 دالة DayOfWeekOrder(int day, int month, int year)

حساب ترتيب اليوم من الأسبوع باستخدام خوارزمية زيلر (Zeller's Congruence):

$(day + y + (y / 4) - (y / 100) + (y / 400) + ((31 * m) / 12)) \% 7$

الناتج من 0 إلى 6 (Sun, 1 = Mon, ..., 6 = Sat = 0)

5 دالة DayShortName(int dayOrder)

تُرجع اختصار اسم اليوم مثل:

"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"

6 دالة MonthShortName(int monthNumber)

تُرجع اختصار اسم الشهر مثل:

"Jan", "Feb", "Mar", ..., "Dec"

7 دالة PrintMonthCalendar(int month, int year)

تُطبع أيام الشهر بالتنسيق الصحيح حسب أول يوم يبدأ فيه الشهر:

تحديد أول يوم من الأسبوع

طباعة العناوين (Sun, Mon, Tue, ...)

ترك مسافات قبل بداية الشهر

طباعة الأيام بترتيبها ضمن الأسابيع.

8 دالة PrintYearCalendar(int year)

طباعة كل الشهور في السنة بترتيبها عبر استدعاء PrintMonthCalendar في حلقة:

for (int i = 1; i <= 12; i++)

PrintMonthCalendar(i, year);

مثال:

Please enter a year to check? 1981

Calendar - 1981

Jan						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Feb						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28

...

📌 ملاحظة:

- السنة الكبيسة تؤثر فقط على عدد أيام شهر فبراير (29 بدلاً من 28)
- الأيام تبدأ من الأحد كالיום الأول في الأسبوع
- لا يوجد تحقق من إدخال غير صالح للسنة

🧐 تفسير:

- البرنامج يعتمد على دالة **DayOfWeekOrder** لحساب بداية كل شهر بدقة باستخدام العمليات الرياضية بدلاً من مكتبات الوقت الجاهزة

✨ خلاصة:

- ✓ البرنامج يستخدم:
- التكرار (loop) لطباعة الشهور
- modular functions لتقسيم المهام
- تنسيق مخرجات أنيق باستخدام **printf**

Problem #10: Days from the Beginning of Year

التعريف: 🔍

الهدف برنامج لحساب عدد الأيام منذ بداية السنة حتى تاريخ معين يُدخله المستخدم (اليوم، الشهر، السنة)

الشرح: 📖

- البرنامج يعتمد على معرفة عدد الأيام في كل شهر
- يجمع الأيام لجميع الأشهر السابقة للشهر المطلوب، ثم يضيف عدد الأيام في الشهر الحالي
- يأخذ في الاعتبار السنوات الكبيسة (Leap Years) لتعديل عدد أيام فبراير إلى 29 بدلاً من 28

الإجراءات: 📌

1 قراءة القيم من المستخدم

`ReadYear()` إدخال السنة

`ReadMonth()` إدخال الشهر

`ReadDay()` إدخال اليوم

2 التحقق من السنة الكبيسة

باستخدام الدالة `bool IsLeapYear(int year)`

تتحقق من كون السنة كبيسة عبر الشرط:

`(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)`

3 حساب عدد الأيام في الشهر

باستخدام

`(int NumberOfDaysInAMonth(int month, int year)`

تُعيد 29 ليوم فبراير إذا كانت السنة كبيسة

4 حساب عدد الأيام منذ بداية السنة

عبر جمع أيام الأشهر السابقة مع اليوم الحالي:

`(int NumberOfDaysFromTheBeginningOfTheYear(int day, int month, int year)`

مثال: 💡

Please enter a Day? 20

Please enter a Month? 5

Please enter a Year? 1981

Number of Days from the beginning of the year is 140

ملاحظة: 📌

- يتم الحساب بافتراض أن الأشهر السابقة صحيحة القيم
- في حال إدخال شهر خارج النطاق (1-12)، تُعيد الدالة صفر

خلاصة: ✨

- البرنامج يجمع أيام الأشهر السابقة ليُظهر كم يوماً مضى منذ بداية السنة حتى اليوم المحدد
- يأخذ بعين الاعتبار السنوات الكبيسة بدقة

Problem #11: Date From Day Order In Year

التعريف: 🔍

الهدف برنامج يحسب عدد الأيام منذ بداية السنة لتاريخ معين، ثم يعيد تحويل هذا الرقم إلى التاريخ نفسه (يوم/شهر/سنة)

الشرح: 📖

البرنامج يقوم بخطوتين رئيسيتين:

الخطوة الأولى: حساب عدد الأيام منذ بداية السنة باستخدام

NumberOfDaysFromTheBeginningOfTheYear

الخطوة الثانية: تحويل هذا الرقم مرة أخرى إلى تاريخ فعلي باستخدام

GetDateFromDayOrderInYear

يستخدم البرنامج السنة الكبيسة لتحديد أيام فبراير بدقة

الإجراءات: 📦

1 قراءة القيم من المستخدم

ReadYear() إدخال السنة

ReadMonth() إدخال الشهر

ReadDay() إدخال اليوم

2 التحقق من السنة الكبيسة

باستخدام الدالة

`bool IsLeapYear(int year)`

3 حساب عدد الأيام منذ بداية السنة

باستخدام الدالة

`int NumberOfDaysFromTheBeginningOfTheYear(int day, int month, int year)`

4 تحويل الرقم إلى تاريخ

باستخدام الهيكل `sDate` والدالة

`sDate GetDateFromDayOrderInYear(int dateOrderInYear, int year)`

تقوم الدالة بطرح أيام كل شهر من الرقم حتى الوصول للشهر الصحيح ثم تحديد اليوم

مثال: 💡

Please enter a Day? 20

Please enter a Month? 5

Please enter a Year? 1981

Number of Days from the beginning of the year is 140

Date for [140] is: 20/5/1981

ملاحظة: 📌

البرنامج دقيق سواء كانت السنة كبيسة أم لا

الهيكل `sDate` يستخدم لتخزين التاريخ الناتج (اليوم، الشهر، السنة)

خلاصة: ✨

✓ البرنامج يربط بين رقم اليوم في السنة والتاريخ الفعلي وهو مناسب لحساب وتحويل أي تاريخ داخل السنة بدقة

Problem #12: Add Days to Date

التعريف:

الهدف برنامج يقوم بقراءة تاريخ محدد وعدد أيام، ثم يعرض التاريخ الناتج بعد إضافة هذه الأيام

الشرح:

البرنامج يعتمد على الهيكل **sDate** لتخزين التاريخ (اليوم، الشهر، السنة)
العملية الأساسية تتم عبر جمع الأيام المراد إضافتها مع عدد الأيام منذ بداية السنة للتاريخ الأصلي، ثم إعادة تحويل الرقم إلى تاريخ كامل

آلية العمل:

- حساب الأيام الإجمالية منذ بداية السنة للتاريخ الأصلي
- إضافة الأيام المراد إضافتها
- تحويل العدد الإجمالي للأيام إلى يوم وشهر وسنة، مع مراعاة السنة الكبيسة والشهور المختلفة

الإجراءات:

- قراءة التاريخ الأصلي من المستخدم
باستخدام الدالة **ReadFullDate()** التي تستدعي:
ReadDay() يوم
ReadMonth() شهر
ReadYear() سنة
- قراءة عدد الأيام المراد إضافتها
باستخدام الدالة **ReadDaysToAdd()**
- حساب التاريخ الجديد بعد إضافة الأيام
باستخدام الدالة **DateAddDays(days, date)**
تتحقق الدالة من عدد أيام كل شهر وتنتقل للسنة التالية عند انتهاء الأشهر

مثال:

Please enter a Day? 20
Please enter a Month? 5
Please enter a Year? 1981
Please enter a Days to add? 3000
Number of Days from the beginning of the year is 140
Date after adding [3000] days is: 14/8/2029

ملاحظة:

البرنامج يدعم السنة الكبيسة
يمكن إضافة أي عدد من الأيام سواء كان صغيراً أم كبيراً (حتى عدة آلاف)
الهيكل **sDate** يجعل التعامل مع التاريخ أكثر وضوحاً وسهولة في البرمجة

خلاصة:

البرنامج يسمح بحساب التاريخ المستقبلي بعد أي عدد من الأيام بدقة وهو مناسب لحساب مواعيد مستقبلية أو مهام زمنية

Problem #13: Date1 Less Than Date2

التعريف:

الهدف برنامج يتحقق مما إذا كان التاريخ الأول (Date1) يسبق التاريخ الثاني (Date2) من حيث الزمن

الشرح:

البرنامج يعتمد على بنية بيانات (struct) لتخزين كل تاريخ على شكل ثلاثة عناصر: اليوم، الشهر، السنة
بعد إدخال التاريخين من المستخدم، تتم مقارنة السنوات أولاً، ثم الأشهر، وأخيراً الأيام لتحديد أيهما أقدم

طريقة المقارنة في الدالة IsDate1BeforeDate2:

1 إذا كانت سنة التاريخ الأول أصغر → يكون التاريخ الأول أقدم

2 إذا كانت السنتان متساويتين → يتم مقارنة الأشهر

3 إذا كانت الأشهر أيضاً متساوية → تتم مقارنة الأيام

الإجراءات:

1 قراءة التاريخ الأول:

من خلال الدالة ReadFullDate() التي تستدعي قراءة اليوم، الشهر، والسنة

2 قراءة التاريخ الثاني:

بنفس الطريقة السابقة

3 إجراء المقارنة:

باستخدام IsDate1BeforeDate2(date1, date2) لتحديد أي التاريخين أسبق

4 عرض النتيجة:

طباعة رسالة توضح ما إذا كان التاريخ الأول أقل (أقدم) من الثاني أم لا

مثال:

Please enter a Day? 12
Please enter a Month? 2
Please enter a Year? 2022
Please enter a Day? 12
Please enter a Month? 3
Please enter a Year? 2022
Yes, Date1 is Less than Date2.

ملاحظة:

المقارنة دقيقة لأنها تبدأ بالسنة وتنتهي باليوم

الدالة IsDate1BeforeDate2 تعيد قيمة منطقية (true / false)

لا يتحقق البرنامج من صحة التاريخ (مثل 30 فبراير)

خلاصة:

✓ البرنامج يتيح التحقق من ترتيب زمني بين تاريخين بدقة
✓ الأسلوب المستخدم بسيط ومنطقي، ويُعد أساساً لتطبيقات مثل فرز التواريخ أو حساب الفروقات الزمنية

Problem #14: Date1 Equals to Date2

التعريف:

الهدف برنامج يقوم بقراءة تاريخين من المستخدم (اليوم، الشهر، السنة) ثم يتحقق إذا كان التاريخ الأول يساوي التاريخ الثاني تمامًا

الشرح:

- البرنامج يستخدم **struct** لتجميع مكونات التاريخ (اليوم، الشهر، السنة) في بنية واحدة
- يتم إدخال كل من اليوم والشهر والسنة عبر دوال منفصلة
- بعد إدخال التاريخين، يتم التحقق من تساوي القيم الثلاثة لكل منهما:
 - السنة
 - الشهر
 - اليوم
- إذا كانت القيم الثلاثة متطابقة، يعتبر التاريخان متساويين

الإجراءات:

- تعريف الهيكل **sDate** الذي يحتوي على **Year**، **Month**، و **Day**.
- إنشاء دوال لقراءة كل من اليوم، الشهر، والسنة
- إنشاء دالة **ReadFullDate()** لقراءة التاريخ بالكامل
- إنشاء دالة **IsDate1EqualDate2()** للمقارنة بين التاريخين
- في الدالة **main()**:
 - قراءة **date1** و **date2**
 - استدعاء دالة المقارنة
 - طباعة النتيجة المناسبة

مثال:

Please enter a Day? 1
Please enter a Month? 1
Please enter a Year? 2000
Please enter a Day? 1
Please enter a Month? 1
Please enter a Year? 2000
Yes, Date1 is Equal to Date2.

تفسير:

- المقارنة تتم بالتسلسل:
- إذا كانت السنة متساوية، ننتقل للمقارنة بين الشهر
- إذا كان الشهر متساويًا أيضًا، نتحقق من اليوم
- إذا تساوت جميعها → التاريخان متساويان

ملاحظة:

يمكن تبسيط دالة المقارنة باستخدام التعبير المنطقي التالي:

```
return (date1.Year == date2.Year &&  
        date1.Month == date2.Month &&  
        date1.Day == date2.Day);
```

✨ خلاصة:

- ✓ البرنامج يتحقق من تساوي تاريخين من حيث اليوم، الشهر، والسنة
- ✓ يستخدم **struct** ودوال منظمة لزيادة وضوح الكود

Problem #15: Last Day, Last Month

التعريف: 🔍

الهدف برنامج يقوم بقراءة تاريخ (يوم، شهر، سنة) ثم يتحقق:

هل اليوم هو آخر يوم في الشهر؟

هل الشهر هو آخر شهر في السنة؟

الشرح: 📖

البرنامج يستخدم هيكل **struct** لتخزين التاريخ (اليوم، الشهر، السنة)
ثم يقرأ القيم من المستخدم باستخدام دوال منفصلة لكل من اليوم والشهر والسنة
بعدها يتم التحقق من:
إذا كانت السنة كبيسة (Leap Year)
وعدد الأيام في الشهر
وأخيراً، تتم مقارنة اليوم والشهر لمعرفة إن كانا في نهاية الشهر أو السنة.

الإجراءات: 📌

- 1 تعريف الهيكل **sDate** الذي يحتوي على **Day, Year, Month**
- 2 إنشاء دوال لإدخال اليوم، الشهر، والسنة من المستخدم
- 3 إنشاء دالة **IsLeapYear()** للتحقق إذا كانت السنة كبيسة
- 4 إنشاء دالة **NumberOfDaysInAMonth()** لتحديد عدد الأيام في كل شهر
- 5 إنشاء دالة **IsLastDayInMonth()** للتحقق إن كان اليوم هو آخر يوم في الشهر
- 6 إنشاء دالة **IsLastMonthInYear()** للتحقق إن كان الشهر هو ديسمبر
- 7 في **main()**:

قراءة التاريخ من المستخدم
التحقق من كونه آخر يوم في الشهر
التحقق من كونه آخر شهر في السنة
طباعة النتائج المناسبة

مثال: 💡

Please enter a Day? 31

Please enter a Month? 12

Please enter a Year? 2022

Yes, Day is Last Day in Month.

Yes, Month is Last Month in Year.

تفسير: 🤖

البرنامج يعتمد على منطق واضح:
اليوم الأخير في الشهر هو عندما يساوي عدد الأيام الكلي في ذلك الشهر
الشهر الأخير في السنة هو عندما يساوي 12

ملاحظة: 📌

الشهر الثاني (فبراير) له 29 يوماً في السنة الكبيسة، و 28 في العادية

دالة **IsLeapYear()** تحدد ذلك تلقائيًا
الدالة **NumberOfDaysInAMonth()** تتعامل مع السنة الكبيسة بشكل صحيح، حيث يصبح فبراير يحتوي على 29 يومًا بدلاً من 28
يمكن إعادة استخدام هذه الدوال في برامج أخرى تتعلق بالتواريخ مثل حساب اليوم التالي أو عدد الأيام بين تاريخين

تحذير: ❌

إذا تم إدخال شهر غير صحيح (أقل من 1 أو أكبر من 12)، فالدالة **NumberOfDaysInAMonth** تعيد القيمة 0

خلاصة: ✨

- ✓ البرنامج يتحقق من اليوم الأخير في الشهر ومن الشهر الأخير في السنة
- ✓ يعتمد على منطق بسيط ودوال مساعدة
- ✓ البرنامج يربط بين المنطق الزمني والتحقق الشرطي
- ✓ استخدام الدوال المنفصلة يجعل الكود منظمًا وسهل القراءة
- ✓ مفيد لفهم التحقق من الشروط المركبة في البرمجة باستخدام **++C**

Problem #16: Increase Date By One Day

التعريف:

الهدف برنامج يقوم بقراءة تاريخ (يوم، شهر، سنة) من المستخدم، ثم يزيد هذا التاريخ بيوم واحد فقط ويعرض النتيجة على الشاشة.

الشرح:

الهدف هو التعامل الصحيح مع تغيّر الأيام، خصوصًا عند الانتقال من نهاية الشهر إلى بداية الشهر الجديد، أو من نهاية السنة إلى السنة التالية
يعتمد البرنامج على مجموعة دوال منظمة لتقسيم المهام مثل قراءة التاريخ، التحقق من السنة الكبيسة، وحساب عدد الأيام في الشهر

الإجراءات:

1 تعريف الهيكل **sDate** لتخزين المعلومات التالية:

Year — السنة

Month — الشهر

Day — اليوم

2 قراءة التاريخ من المستخدم عبر دوال:

ReadDay()

ReadMonth()

ReadYear()

3 التحقق من السنة الكبيسة:

IsLeapYear()

تعيد **true** إذا كانت السنة كبيسة، أي تقبل القسمة على 4 ولا على 100، أو تقبل القسمة على 400

4 معرفة عدد الأيام في كل شهر:

NumberOfDaysInAMonth()

تحدد عدد الأيام حسب الشهر والسنة (فبراير = 28 أو 29 في حال السنة الكبيسة)

5 التحقق من اليوم الأخير في الشهر:

IsLastDayInMonth()

تتحقق إن كان اليوم الحالي هو آخر يوم في الشهر

6 التحقق من الشهر الأخير في السنة:

IsLastMonthInYear()

تعيد **true** إذا كان الشهر هو ديسمبر (12)

7 زيادة التاريخ بيوم واحد:

IncreaseDateByOneDay()

إذا كان اليوم الأخير في الشهر والأخير في السنة:

يتم زيادة السنة +1، وتعيين الشهر واليوم إلى 1

إذا كان اليوم الأخير فقط في الشهر (وليس في السنة):

يتم زيادة الشهر +1 وإعادة اليوم إلى 1

غير ذلك:

يتم فقط زيادة اليوم +1

8 عرض النتيجة النهائية:

طباعة التاريخ الجديد بعد زيادة يوم واحد

مثال: 💡

Please enter a Day? 31

Please enter a Month? 12

Please enter a Year? 2022

Date after adding one day is: 1/1/2023

ملاحظة: 📌

البرنامج يتعامل بدقة مع الانتقال بين الأشهر والسنوات، بما في ذلك السنة الكبيسة
الكود قابل للتوسيع ليزيد بعدد أيام أكبر باستخدام حلقة تستدعي نفس الدالة عدة مرات

تفسير: 🤖

البرنامج يربط بين منطق التقويم الميلادي وقواعده الرياضية:
عندما ينتهي عدد أيام الشهر، يبدأ العدّ من جديد ويزداد الشهر أو السنة حسب الحاجة

تحذير: 🚫

إذا تم إدخال شهر خارج المدى (1-12) أو يوم غير صالح، لن يعطي البرنامج نتيجة دقيقة لأنه لا يحتوي على معالجة أخطاء الإدخال

خلاصة: ✨

البرنامج يزيد أي تاريخ بيوم واحد بدقة ✓
يأخذ في الحسبان الأيام الأخيرة في الأشهر والسنة الكبيسة ✓

Problem #17: Difference in Days Between Two Dates

التعريف:

الهدف برنامج يقوم بقراءة تاريخين (Date1 و Date2) من المستخدم، ثم يحسب الفرق بينهما بعدد الأيام، مع إمكانية حساب الفرق بشمول اليوم الأخير أو بدونه

الشرح:

الهدف من البرنامج هو معرفة عدد الأيام بين تاريخين محددين بطريقة دقيقة تشمل الانتقال بين الأشهر والسنوات، مع مراعاة السنوات الكبيسة

الإجراءات:

1) تعريف الهيكل sDate:

يحتوي على:

Year — السنة

Month — الشهر

Day — اليوم

2) دوال قراءة المدخلات من المستخدم:

ReadDay()

ReadMonth()

ReadYear()

ReadFullDate() — تجمع الثلاثة معًا في متغير من نوع sDate

3) التحقق من السنة الكبيسة:

IsLeapYear()

تعيد true إذا كانت السنة تقبل القسمة على 4 ولا على 100، أو تقبل القسمة على 400

4) مقارنة تاريخين:

IsDate1BeforeDate2()

تعيد true إذا كان Date1 أقدم من Date2 (أي قبلها زمنيًا)

5) معرفة عدد الأيام في الشهر:

NumberOfDaysInAMonth()

تحدد عدد الأيام حسب الشهر والسنة، مع التعامل الصحيح مع فبراير في السنة الكبيسة

6) زيادة التاريخ بيوم واحد:

IncreaseDateByOneDay()

إذا كان اليوم الأخير في الشهر الأخير من السنة → ينتقل إلى 1/1 في السنة التالية

إذا كان اليوم الأخير في الشهر فقط → ينتقل إلى اليوم 1 من الشهر التالي

غير ذلك → زيادة اليوم فقط

7) حساب الفرق بالأيام بين تاريخين:

GetDifferenceInDays(date1, date2, includeEndDay)

تستخدم حلقة while لتزيد date1 يومًا في كل مرة حتى تصل إلى date2

كل زيادة تعني مرور يوم واحد، ويُجمع في العداد days

عند تفعيل includeEndDay = true يتم إضافة يوم إضافي في النتيجة

مثال:

Please enter a Day? 8

Please enter a Month? 6

Please enter a Year? 1980

Please enter a Day? 20

Please enter a Month? 5

Please enter a Year? 1981

Difference is: 346 Day(s).

Difference (Including End Day) is: 347 Day(s).

ملاحظة: 

يجب أن يكون التاريخ الأول (Date1) أقدم من الثاني (Date2)، وإلا سيعطي البرنامج صفرًا 
البرنامج يعتمد على التكرار اليومي، لذا قد يكون أبطأ قليلاً عند حساب فترات زمنية طويلة جدًا، لكنه أكثر دقة وبساطة 

تفسير: 

البرنامج يحاكي مرور الأيام فعليًا من تاريخ إلى آخر، بدلاً من الحساب الرياضي المباشر، مما يضمن الدقة حتى عند التبديل بين أشهر وسنوات كبيسة 

تحذير: 

إذا تم إدخال تاريخ غير صحيح (مثل شهر 13 أو يوم 32)، لن يتحقق البرنامج من صحة الإدخال وقد يعطي نتيجة غير منطقية 

خلاصة: 

✓ البرنامج يحسب الفرق بين تاريخين بدقة عالية
✓ يأخذ بالحسبان السنوات الكبيسة وتبديل الأشهر

Problem #18: Your Age in Days

التعريف:

الهدف برنامج يقوم بحساب **عمر المستخدم بالأيام** بناءً على تاريخ ميلاده وتاريخ اليوم الحالي

الإجراءات:

- 1 تعريف الهيكل `sDate`:
يحتوي على ثلاث قيم: `Day`، `Month`، `Year` لتمثيل التاريخ
- 2 قراءة التاريخ من المستخدم:
ثلاث دوال لقراءة اليوم، الشهر، السنة (`ReadDay`، `ReadMonth`، `ReadYear`)
ثم تُستخدم داخل `ReadFullDate` لتكوين تاريخ كامل
- 3 التحقق من السنة الكبيسة:
باستخدام دالة `IsLeapYear(year)` التي ترجع `true` إذا كانت السنة كبيسة
- 4 معرفة عدد الأيام في الشهر:
عبر دالة `NumberOfDaysInAMonth(month, year)` التي تراعي شباط (28 أو 29 يوماً)
- 5 التحقق من آخر يوم أو آخر شهر:
`IsLastDayInMonth(date)` و `IsLastMonthInYear(month)` تساعدان في الانتقال الصحيح بين الأشهر والسنوات
- 6 زيادة التاريخ بيوم واحد:
الدالة `IncreaseDateByOneDay(date)` تتعامل مع الحالات المختلفة (نهاية الشهر أو السنة)
- 7 مقارنة تاريخين:
`IsDate1BeforeDate2(date1, date2)` لمعرفة أي التاريخين أسبق
- 8 حساب الفرق بين تاريخين:
`GetDifferenceInDays(date1, date2, includeEndDay)` تحسب عدد الأيام بين تاريخ الميلاد وتاريخ اليوم
- 9 الحصول على تاريخ النظام الحالي:
باستخدام `GetSystemDate()` لجلب التاريخ من ساعة النظام
- 10 البرنامج الرئيسي `main`:
يقرأ تاريخ الميلاد
يجلب التاريخ الحالي
يعرض النتيجة بالأيام

مثال:

Please Enter Your Date of Birth:

Please enter a Day? 20

Please enter a Month? 5

Please enter a Year? 1981

Your Age is : 16233 Day(s).

خلاصة:

- ✓ البرنامج يعتمد على حساب الفرق اليومي بين تاريخين
- ✓ يستخدم مفهوم الزيادة اليومية والتواريخ الكبيسة لضمان دقة عالية
- ✓ مفيد لتطبيقات تتعامل مع الزمن مثل الأعمار أو حساب فترات العمل

Problem #19: Diff In Days (Negative Days)

التعريف:

الهدف برنامج لحساب الفرق بين تاريخين (بالأيام) مع مراعاة إمكانية أن يكون الفرق سالبًا إذا كان التاريخ الثاني أقدم من الأول

الشرح:

الهدف برنامج يقرأ تاريخين من المستخدم (اليوم، الشهر، السنة) ويحسب الفرق بينهما بالأيام
إذا كان التاريخ الثاني أقدم، تُطبع النتيجة بالسالب
كما يمكن حساب الفرق مع أو بدون تضمين اليوم الأخير

الإجراءات:

1 تعريف الهيكل:

هيكل **sDate** يحتوي على:

Day, Month, Year

2 إدخال البيانات:

دوال لقراءة اليوم، الشهر، السنة وتجميعها في متغير من نوع **sDate**

3 التحقق من السنة الكبيسة:

IsLeapYear() تتحقق إن كانت السنة كبيسة لتعديل عدد أيام فبراير

4 مقارنة التاريخين:

IsDate1BeforeDate2() لتحديد من الأقدم

5 عدد الأيام في الشهر:

NumberOfDaysInAMonth() لإرجاع عدد الأيام مع مراعاة فبراير والسنة الكبيسة

6 زيادة اليوم:

IncreaseDateByOneDay() لزيادة التاريخ يومًا واحدًا مع الانتقال للشهر أو السنة التالية عند الحاجة

7 التبديل بين التاريخين:

SwapDates() إذا كان التاريخ الأول أحدث، يتم التبديل ليكون الأول هو الأقدم

8 حساب الفرق:

GetDifferenceInDays() تحسب عدد الأيام بين التاريخين

إذا كان **includeEndDay = true** تُضاف يوم النهاية للفرق

مثال:

Date1 = 1/1/2000

Date2 = 1/1/2022

Difference is: 8036 Day(s)

Difference (Including End Day): 8037 Day(s)

Date1 = 1/1/2022

Date2 = 1/1/2000

Difference is: -8036 Day(s)

Difference (Including End Day): -8037 Day(s)

ملاحظة:

البرنامج يعيد الفرق بالسالب تلقائيًا عند عكس التواريخ بفضل دالة **SwapDates()**

الحساب يتم عبر تكرار زيادة اليوم حتى الوصول للتاريخ الثاني

✨ خلاصة:

- ✓ البرنامج يربط بين الحسابات الزمنية والمقارنة المنطقية للتواريخ
- ✓ يوضح مفهوم الـ **iteration** عبر زيادة الأيام يدويًا
- ✓ يدعم الفرق السالب وتضمنين يوم النهاية بشكل مرن

Problem #20 – #32: Increase Date Problems

التعريف:

الهدف البرنامج يقرأ تاريخاً من المستخدم ثم يزيده بعدد محدد من الأيام، الأسابيع، الأشهر، السنوات، العقود، القرون أو الألفيات باستخدام دوال منفصلة لكل نوع من الزيادة

الإجراءات:

1 تعريف الهيكل sDate:

يحتوي على ثلاث قيم: `Year`, `Month`, `Day`

2 قراءة المدخلات:

عبر الدوال:

`ReadDay()`

`ReadMonth()`

`ReadYear()`

`ReadFullDate()` وتجميعها في

3 التحقق من السنة الكبيسة:

الدالة `IsLeapYear(year)` تحدد ما إذا كانت السنة تحتوي على 29 يوماً في فبراير

4 عدد الأيام في كل شهر:

`NumberOfDaysInAMonth(month, year)` تعيد العدد الصحيح بناءً على الشهر والسنة

5 زيادة اليوم الواحد:

`IncreaseDateByOneDay(date)` تزيد اليوم، مع مراعاة الانتقال إلى الشهر أو السنة الجديدة

6 زيادة بعدد معين من الأيام:

`IncreaseDateByXDays(days, date)` تستدعي الدالة السابقة بعدد المرات المطلوبة

7 زيادة أسبوع أو عدة أسابيع:

`IncreaseDateByOneWeek()` تزيد 7 أيام

`IncreaseDateByXWeeks(weeks, date)` يفترض أن يزيد `weeks*7` (لكن الكود الحالي يزيد يوماً

لكل أسبوع فقط !)

ملاحظة: يوجد خطأ منطقي بسيط: يجب ضرب عدد الأسابيع $\times 7$ داخل الحلقة

8 زيادة شهر أو عدة أشهر:

`IncreaseDateByOneMonth()` تضيف شهراً واحداً مع التأكد أن اليوم لا يتجاوز الحد الأقصى للشهر الجديد

`IncreaseDateByXMonths(months, date)` تستدعي السابقة عدة مرات

9 زيادة سنة أو أكثر:

`IncreaseDateByOneYear()` تضيف سنة مع تعديل يوم 29 فبراير إلى 28 عند الحاجة

`IncreaseDateByXYears()` تكرر العملية

`IncreaseDateByXYearsFaster()` تضيف السنوات مباشرة في خطوة واحدة

العقد (Decade):

`IncreaseDateByOneDecade()` تضيف 10 سنوات

`IncreaseDateByXDecades()` نضيف (decade $\times 10$) سنوات ببطء (تدرجياً)

`IncreaseDateByXDecadesFaster()` تضيفها مباشرة في خطوة واحدة

القرن (Century):

`IncreaseDateByOneCentury()` تضيف 100 سنة

الألفية (Millennium):

`IncreaseDateByOneMillennium()` تضيف 1000 سنة

مثال: 💡

Please enter a Day? 31

Please enter a Month? 12

Please enter a Year? 2022

Date After:

01-Adding one day	is: 1/1/2023
02-Adding 10 days	is: 11/1/2023
03-Adding one week	is: 18/1/2023
04-Adding 10 weeks	is: 29/3/2023
05-Adding one month	is: 29/4/2023
06-Adding 5 months	is: 29/9/2023
07-Adding one year	is: 29/9/2024
08-Adding 10 Years	is: 29/9/2034
09-Adding 10 Years (faster)	is: 29/9/2044
10-Adding one Decade	is: 29/9/2054
11-Adding 10 Decades	is: 29/9/2154
12-Adding 10 Decade (faster)	is: 29/9/2254
13-Adding One Century	is: 29/9/2354
14-Adding One Millennium	is: 29/9/3354

خلاصة: ✨

✓ البرنامج يُظهر قوة البرمجة المهيكلية باستخدام الدوال
✓ يمكن تطويره بسهولة لحساب فترات زمنية أخرى أو تقويمات مختلفة

Problem #33 – #46: Decrease Date Problems

التعريف:

الهدف البرنامج يقرأ تاريخاً من المستخدم ثم يُنقصه بمقدار محدد من الأيام، الأسابيع، الأشهر، السنوات، العقود، القرون أو الألفيات، مع مراعاة منطق الانتقال بين الشهور والسنوات بشكل صحيح

الإجراءات:

1 تعريف الهيكل sDate:

يحتوي على ثلاثة عناصر: 'Year', 'Month', 'Day' لتمثيل التاريخ

2 قراءة المدخلات من المستخدم:

ReadDay() – قراءة اليوم

ReadMonth() – قراءة الشهر

ReadYear() – قراءة السنة

جميعها تُدمج في الدالة ReadFullDate() لإرجاع التاريخ الكامل

3 التحقق من السنة الكبيسة:

الدالة IsLeapYear(year) تتحقق ما إذا كانت السنة كبيسة (أي تحتوي فبراير على 29 يوماً)

4 عدد الأيام في كل شهر:

NumberOfDaysInAMonth(month, year) تعيد العدد الصحيح لأيام الشهر مع مراعاة السنة الكبيسة

5 إنقاص يوم واحد:

DecreaseDateByOneDay(date) تقلل اليوم بمقدار واحد، وتنتقل تلقائياً إلى الشهر أو السنة السابقة عند الوصول إلى بداية الشهر أو السنة

مثال: من '1/1/2023' → '31/12/2022'

6 إنقاص عدد معين من الأيام أو الأسابيع:

DecreaseDateByXDays(days, date) تكرر إنقاص يوم بعدد الأيام المطلوبة

DecreaseDateByOneWeek(date) تقلل 7 أيام

DecreaseDateByXWeeks(weeks, date) تكرر العملية بعدد الأسابيع المطلوبة

7 إنقاص شهر أو عدة أشهر:

DecreaseDateByOneMonth(date) تقلل شهراً واحداً مع تعديل اليوم في حال لم يوجد (مثل الانتقال من

31 مارس إلى 28 فبراير)

DecreaseDateByXMonths(months, date) تكرر العملية لعدة أشهر

8 إنقاص سنة أو أكثر:

DecreaseDateByOneYear(date) تقلل سنة واحدة وتتعامل مع 29 فبراير في السنوات غير الكبيسة

DecreaseDateByXYears(years, date) تكرر العملية سنة بعد سنة

DecreaseDateByXYearsFaster(years, date) تقلل السنوات مباشرة في خطوة واحدة

9 العقود، القرون، والألفيات:

DecreaseDateByOneDecade() تقلل 10 سنوات

DecreaseDateByXDecades() تقلل (decade × 10) سنة بطريقة متكررة

DecreaseDateByXDecadesFaster() تقللها دفعة واحدة

DecreaseDateByOneCentury() تقلل 100 سنة

DecreaseDateByOneMillennium() تقلل 1000 سنة

مثال:

Please enter a Day? 31

Please enter a Month? 12

Please enter a Year? 2022

Date After:

01: Subtracting one day	is: 30/12/2022
02: Subtracting 10 days	is: 20/12/2022
03: Subtracting one week	is: 13/12/2022
04: Subtracting 10 weeks	is: 04/10/2022
05: Subtracting one month	is: 04/09/2022
06: Subtracting 5 months	is: 04/04/2022
07: Subtracting one year	is: 04/04/2021
08: Subtracting 10 years	is: 04/04/2011
09: Subtracting 10 years (faster)	is: 04/04/2001
10: Subtracting one Decade	is: 04/04/1991
11: Subtracting 10 Decades	is: 04/04/1891
12: Subtracting 10 Decades (faster)	is: 04/04/1791
13: Subtracting one Century	is: 04/04/1691
14: Subtracting one Millennium	is: 04/04/0691

ملاحظة: 

الدوال "السريعة" (Faster) تُجري الحسابات مباشرة على السنة لتوفير الوقت بدل التكرار 
البرنامج يحافظ على صحة اليوم حتى عند الانتقال إلى شهر بأيام أقل 

مقارنة: 

النوع	الطريقة	الأداء
بطيئة (loop)	تكرر العملية عدة مرات	دقيقة لكن أبطأ
سريعة (direct)	تقلل السنوات مباشرة	أسرع لكن تتطلب تحققًا إضافيًا من 29 فبراير

خلاصة: 

- ✓ البرنامج يُظهر تطبيقًا متكاملًا لمفاهيم التعامل مع التواريخ
- ✓ يغطي جميع الوحدات الزمنية الممكنة للإنقاص
- ✓ يحافظ على منطقية التاريخ بشكل دقيق عند الانتقال بين الأشهر والسنوات

Problem #47 – #53: More Date Problems

التعريف:

- الهدف برنامج بلغة C++ يقرأ تاريخاً (يوم، شهر، سنة) ويحسب مجموعة من المعلومات المرتبطة بالتاريخ مثل:
 - اليوم من الأسبوع
 - هل هو نهاية أسبوع أو عمل
 - وعدد الأيام المتبقية لنهاية الأسبوع، الشهر، والسنة

الشرح:

- الهدف برنامج يستخدم هيكل (struct) لتخزين التاريخ:

```
struct sDate
{
    short Year;
    short Month;
    short Day;
};
```

الإجراءات:

- 1 قراءة التاريخ من المستخدم
يحتوي `ReadFullDate()`, `ReadDay()`, `ReadMonth()`, `ReadYear()` و `sDate` هذه الدوال تقرأ اليوم، الشهر، والسنة وتعيد كائن `sDate`
- 2 التحقق من سنة كبيسة
`IsLeapYear(year)`
ترجع `true` إذا كانت السنة كبيسة (تقسم على 4 وليس على 100 إلا إذا كانت على 400)
- 3 عدد الأيام في الشهر
`NumberOfDaysInAMonth(month, year)`
تستخدم للتحقق من نهاية الشهر
- 4 حساب اليوم من الأسبوع
`DayOfWeekOrder(day, month, year)`
تعيد رقم اليوم من الأسبوع (0=الأحد، 6=السبت)
`DayShortName()` تعيد الاسم المختصر مثل (Sun, Mon, ...)
- 5 التحقق من نوع اليوم
`IsEndOfWeek(date)` هل هو السبت؟
`IsWeekEnd(date)` هل اليوم جمعة أو سبت؟
`IsBusinessDay(date)` → هل اليوم من الأحد إلى الخميس؟
- 6 زيادة التاريخ بيوم واحد
`IncreaseDateByOneDay(date)`
تستخدم لحساب الفروقات بين تاريخين
- 7 حساب الفروقات الزمنية
`GetDifferenceInDays(date1, date2)`
تعيد عدد الأيام بين تاريخين
- 8 الأيام حتى نهاية الأسبوع / الشهر / السنة
`DaysUntilTheEndOfWeek(date)`
`DaysUntilTheEndOfMonth(date)`
`DaysUntilTheEndOfYear(date)`

مثال: 💡

Today is Fri, 23/9/2022

Is it End of Week?

No Not end of week.

Is it Weekend?

Yes it is a week end.

Is it Business Day?

No it is NOT a business day.

Days until end of week : 1 Day(s).

Days until end of month : 8 Day(s).

Days until end of year : 100 Day(s).

ملاحظة: 📌

البرنامج يمكنه استخدام تاريخ النظام الحالي تلقائيًا أو قراءة تاريخ من المستخدم يدويًا 🟡

خلاصة: ✨

البرنامج يطبق مفاهيم عملية في إدارة التواريخ مثل: ✓

التحقق من صلاحية التاريخ 🟢

عمليات حسابية على التواريخ 🟢

تحديد نوع اليوم (عمل / عطلة) 🟢

استخدام دوال متعددة لتقسيم المنطق 🟢

Problem #54: Calculate Vacation Days

التعريف:

الهدف برنامج بلغة C++ يقوم بحساب عدد أيام الإجازة الفعلية بين تاريخين (من - إلى) مع استبعاد عطلة نهاية الأسبوع (الجمعة والسبت) من الحساب

الشرح:

يعتمد البرنامج على بنية بيانات **struct sDate** لتمثيل التاريخ، ويتضمن عدة دوال مساعدة لإجراء العمليات الزمنية

الإجراءات:

- 1 قراءة التاريخين من المستخدم
ReadFullDate() تقرأ اليوم، الشهر، والسنة وتُرجع كائن **sDate** لكل من تاريخ البداية وتاريخ النهاية
- 2 التحقق من السنة الكبيسة
IsLeapYear(year) تُستخدم لحساب عدد الأيام في شهر شباط (29 بدلاً من 28 عند الكبيسة)
- 3 تحديد عدد الأيام في الشهر
NumberOfDaysInAMonth(month, year) تُستخدم لمعرفة نهاية الشهر في الدوال اللاحقة
- 4 زيادة التاريخ بيوم واحد
IncreaseDateByOneDay(date) تُستخدم للتنقل بين الأيام أثناء العد في فترة الإجازة
- 5 تحديد يوم الأسبوع
DayOfWeekOrder(day, month, year) تُرجع ترتيب اليوم (0 للأحد ... 6 للسبت)
DayShortName() تُرجع الاسم المختصر لليوم مثل (Sun, Mon, ...)
- 6 التحقق من نوع اليوم
IsWeekEnd(date) يتحقق إن كان اليوم جمعة أو سبت
IsBusinessDay(date) يتحقق إن كان اليوم يوم عمل (الأحد إلى الخميس)
- 7 حساب أيام الإجازة الفعلية
CalculateVacationDays(dateFrom, dateTo) تبدأ من **dateFrom** حتى **dateTo**
لكل يوم يتم التحقق:
إذا كان يوم عمل يتم زيادته في العداد
ثم يتم الانتقال لليوم التالي

مثال:

Vacation Starts:

Please enter a Day? 1

Please enter a Month? 9

Please enter a Year? 2022

Vacation Ends:

Please enter a Day? 5

Please enter a Month? 9

Please enter a Year? 2022

Vaction From: Thu, 1/9/2022

Vaction To: Mon, 5/9/2022

Actual Vacation Days is: 2

تفسير: 🤖

الخميس (1/9) = يوم عمل

الجمعة والسبت عطلة

الأحد والاثنين يومي عمل

المجموع = 2 أيام عمل

ملاحظة: 📌

العدد لا يشمل عطلة نهاية الأسبوع

البرنامج يحسب الأيام بين التاريخين باستثناء اليوم الأخير إذا لم يكن يوم عمل

خلاصة: ✨

البرنامج يطبق مفاهيم: ✓

التعامل مع التواريخ Structs

التحقق من نوع اليوم (عمل / عطلة)

استخدام حلقات وتوابع مساعدة لحساب الفروقات الزمنية

تطبيق عملي لمهارات المنطق الشرطي في البرمجة

Problem #55: Calculate Vacation Return Date

التعريف:

الهدف برنامج بلغة C++ يقوم بحساب تاريخ العودة من الإجازة بناءً على:
تاريخ بداية الإجازة
وعدد أيام الإجازة
مع استبعاد عطلة نهاية الأسبوع (الجمعة والسبت) من العد.

الشرح:

يعتمد البرنامج على بنية **struct sDate** لتمثيل التاريخ (اليوم، الشهر، السنة)، ويستخدم مجموعة من الدوال المساعدة لمعالجة التواريخ بدقة

الإجراءات:

- 1 قراءة بيانات الإجازة من المستخدم
ReadFullDate() لقراءة تاريخ البداية
ReadVacationDays() لقراءة عدد أيام الإجازة
- 2 التحقق من صحة التواريخ والسنوات
IsLeapYear(year) تتحقق إن كانت السنة كبيسة
NumberOfDaysInAMonth(month, year) تعطي عدد الأيام في الشهر (مع الأخذ بعين الاعتبار السنة الكبيسة)
- 3 تقدم التاريخ يوماً بيوم
IncreaseDateByOneDay(date)
إذا وصلنا إلى نهاية الشهر، ينتقل إلى الشهر التالي
وإذا وصلنا إلى نهاية السنة، ينتقل إلى السنة التالية
- 4 تحديد نوع اليوم
DayOfWeekOrder(day, month, year) تعيد ترتيب اليوم في الأسبوع (0 للأحد ... 6 للسبت)
IsWeekEnd(date) يتحقق إن كان اليوم جمعة أو سبت
IsBusinessDay(date) يتحقق إن كان اليوم يوم عمل (الأحد إلى الخميس)
- 5 حساب تاريخ العودة
CalculateVacationReturnDate(dateFrom, vacationDays)
الخطوات المنطقية:
إذا كان تاريخ البداية عطلة نهاية الأسبوع → يتجاهلها ويبدأ من أول يوم عمل
يضيف عدد أيام الإجازة، مع تخطي أيام العطل (الجمعة والسبت)
بعد انتهاء العد، إذا صادف يوم العودة عطلة نهاية الأسبوع → يتقدم إلى أول يوم عمل بعدها

مثال:

Vacation Starts:

Please enter a Day? 1

Please enter a Month? 1

Please enter a Year? 2022

Please enter vacation days? 23

Return Date: Wed , 2/2/2022

🧐 تفسير:

- تبدأ الإجازة من السبت 1/1/2022 (عطلة يتم تجاوزها)
- تبدأ فعلياً من الأحد 2/1/2022
- بعد احتساب 23 يوم عمل (مع استبعاد الجمع والسبت)، تكون العودة في الأربعاء 2/2/2022

📌 ملاحظة:

- البرنامج يتجاهل جميع عطلات نهاية الأسبوع قبل وخلال الإجازة
- العداد يشمل فقط أيام العمل الفعلية
- الإجازة تبدأ فعلياً من أول يوم عمل بعد تاريخ البداية إذا صادف عطلة

✨ خلاصة:

- البرنامج يستخدم: ✓
- الحلقات والدوال الشرطية لمعالجة التواريخ بدقة
- منطق التواريخ الحقيقي مع دعم الانتقال بين الأشهر والسنوات
- دوال متعددة لإعادة الاستخدام وتنظيم الكود

Problem #56: Is Date1 After Date2

التعريف: 🔍

sDate Struct بنية تحتوي على ثلاث قيم: السنة، الشهر، اليوم

الدوال الأساسية:

IsDate1BeforeDate2 تتحقق إن كان التاريخ الأول قبل الثاني

IsDate1EqualDate2 تتحقق إن كان التاريخان متساويين

IsDate1AfterDate2 تتحقق إن كان التاريخ الأول بعد الثاني (أي ليس قبله ولا مساوٍ له)

الإجراءات: 🟦

1 يطلب إدخال اليوم، الشهر، السنة للتاريخ الأول

2 يطلب إدخال اليوم، الشهر، السنة للتاريخ الثاني

3 يجري مقارنة دقيقة حسب الترتيب: السنة الشهر اليوم

4 يطبع النتيجة المناسبة على الشاشة

مثال: 💡

Enter Date1:

Please enter a Day? 1

Please enter a Month? 1

Please enter a Year? 2022

Enter Date2:

Please enter a Day? 1

Please enter a Month? 1

Please enter a Year? 2000

Yes, Date1 is After Date2.

ملاحظة: 📌

المقارنة تبدأ بالسنة أولاً لأنها الأعلى ترتيباً زمنياً

لا يعتمد البرنامج على مكتبات جاهزة للتاريخ بل على منطق رياضي بسيط

خلاصة: ✨

✓ البرنامج يحدد إذا كان **Date1 > Date2** عبر ثلاث دوال منظمة

✓ يعتمد على مقارنة القيم بالتسلسل المنطقي (سنة → شهر → يوم)

✓ لا يستخدم توابع أو مكتبات جاهزة مثل **ctime**

Problem #57: Compare Date Function

التعريف:

- برنامج يقوم بقراءة تاريخين من المستخدم ثم يقارن بينهما ليحدد ما إذا كان
 - التاريخ الأول قبل الثاني
 - التاريخان متساويان
 - التاريخ الأول بعد الثاني

الشرح:

- المقارنة تعتمد على الترتيب الزمني (السنة الشهر اليوم)
- تستخدم دوال منطقية (`IsDate1BeforeDate2`, `IsDate1EqualDate2`, `IsDate1AfterDate2`) لتحديد العلاقة بين التاريخين
- الدالة الأساسية `CompareDates` تُرجع قيمة عددية من نوع `enum` تمثل النتيجة

الإجراءات:

1 تعريف `sDate` struct

- يحتوي على:
 - `Year`
 - `Month`
 - `Day`

2 تعريف `enDateCompare` enum

- التعريف: لتحديد نتيجة المقارنة بين تاريخين
 - `Before = -1`
 - `Equal = 0`
 - `After = 1`

3 الدوال المساعدة لقراءة البيانات

- `ReadDay()` – إدخال اليوم
- `ReadMonth()` – إدخال الشهر
- `ReadYear()` – إدخال السنة
- `ReadFullDate()` – تجمع القيم السابقة في هيكل واحد من نوع `sDate`

4 دالة `IsDate1BeforeDate2`

- التعريف: تتحقق ما إذا كان التاريخ الأول قبل الثاني زمنياً
- المقارنة تتم أولاً حسب السنة، ثم الشهر، ثم اليوم

5 دالة `IsDate1EqualDate2`

- التعريف: تتحقق ما إذا كان التاريخان متطابقين تماماً

6 دالة `IsDate1AfterDate2`

- التعريف: تتحقق إن كان التاريخ الأول بعد الثاني، بشرط ألا يكون قبله أو مساوياً له

7 دالة `CompareDates`

- التعريف: تُعيد القيمة المناظرة للعلاقة بين التاريخين:
 - 1 التاريخ الأول قبل الثاني
 - 0 التاريخان متساويان
 - 1 التاريخ الأول بعد الثاني

8 الدالة الرئيسية `main()`

- تطلب من المستخدم إدخال تاريخين
- تستدعي `CompareDates` وتطبع النتيجة

مثال: 💡

Enter Date1:

Please enter a Day? 1

Please enter a Month? 1

Please enter a Year? 2000

Enter Date2:

Please enter a Day? 1

Please enter a Month? 1

Please enter a Year? 2022

Compare Result = -1

✨ خلاصة:

- ✓ البرنامج يقارن بين تاريخين ويُرجع علاقة زمنية دقيقة
- ✓ يعتمد على الترتيب الهرمي (السنة الشهر اليوم)
- ✓ يستخدم **enum** لجعل الكود أكثر وضوحًا وتنظيمًا

Problem #58: Is Overlap Periods

التعريف:

- sDate Struct**: هيكل يحتوي على اليوم، الشهر، والسنة
- sPeriod Struct**: هيكل يحتوي على تاريخين: ****StartDate**** و ****EndDate**** لتمثيل فترة زمنية
- enDateCompare Enum**: نوع معدد (Enumeration) يحدد نتيجة مقارنة تاريخين:
 - Before = -1**: التاريخ الأول قبل الثاني
 - Equal = 0**: التاريخان متساويان
 - After = 1**: التاريخ الأول بعد الثاني

الشرح:

- هذا البرنامج يقرأ فترتين زمنيتين (**Period1** و **Period2**) ويقوم بالتحقق مما إذا كانت الفترتان تتداخلان (**overlap**) أي يوجد بينهما جزء مشترك في الزمن

الدوال الأساسية:

- CompareDates(date1, date2)**: ترجع إذا كان التاريخ الأول قبل، بعد، أو يساوي الثاني
- IsOverlapPeriods(period1, period2)**: تتحقق من وجود تداخل بين الفترتين حسب المنطق التالي:
 - CompareDates** إذا كانت نهاية الفترة الثانية قبل بداية الأولى لا يوجد تداخل
 - CompareDates** إذا كانت بداية الفترة الثانية بعد نهاية الأولى لا يوجد تداخل
 - غير ذلك يوجد تداخل

الإجراءات:

- 1 قراءة تاريخ بداية ونهاية الفترة الأولى
- 2 قراءة تاريخ بداية ونهاية الفترة الثانية
- 3 استدعاء الدالة **IsOverlapPeriods** للتحقق من التداخل
- 4 طباعة النتيجة المناسبة

مثال:

Period 1: 1/2/2022 10/2/2022
Period 2: 5/2/2022 15/2/2022
Yes, Periods Overlap.

تفسير:

- لأن الفترة الثانية تبدأ داخل نطاق الفترة الأولى (بين 1 و 10 فبراير)، فهناك تداخل واضح

ملاحظة:

- لو كانت إحدى الفترات تنتهي قبل أن تبدأ الأخرى تمامًا، فالنتيجة ستكون **"No Overlap"**

خلاصة:

- ✓ البرنامج يحدد إذا كان هناك تداخل زمني بين فترتين باستخدام المقارنة المنطقية للتواريخ
- ✓ يعتمد على مقارنة البداية والنهاية لكل فترة

❑ لا يعتمد على مكتبات جاهزة بل على حسابات زمنية منطقية مبنية على القيم المدخلة يدويًا

Problem #59: Period Length In Days

التعريف:

sDate Struct يمثل تاريخًا واحدًا يحتوي على اليوم، الشهر، والسنة

sPeriod Struct يمثل فترة زمنية بين تاريخين:

StartDate – تاريخ البداية

EndDate – تاريخ النهاية

الشرح:

هذا البرنامج يقرأ فترة زمنية (تاريخ بداية وتاريخ نهاية) ثم يحسب عدد الأيام بينهما، مع خيار احتساب يوم النهاية ضمن المدة أو استثنائه

الدوال الأساسية:

IsLeapYear(year) تتحقق ما إذا كانت السنة كبيسة (أي تحتوي على 29 يومًا في فبراير)

NumberOfDaysInAMonth(month, year) تُرجع عدد الأيام في شهر معين، مع مراعاة السنة الكبيسة

IncreaseDateByOneDay(date) تزيد اليوم بمقدار يوم واحد مع الانتقال إلى الشهر أو السنة التالية عند الحاجة

GetDifferenceInDays(date1, date2, includeEndDay) تحسب عدد الأيام بين تاريخين عن طريق

تكرار زيادة اليوم حتى الوصول إلى التاريخ الثاني

إذا كان **includeEndDay = true** يتم احتساب يوم النهاية أيضًا

PeriodLengthInDays(period, includeEndDate) تحسب طول الفترة الزمنية بناءً على تواريخ البداية

والنهاية

الإجراءات:

1 قراءة تاريخ البداية للفترة

2 قراءة تاريخ النهاية للفترة

3 حساب الفرق بين التاريخين باستخدام **PeriodLengthInDays()**

4 طباعة المدة مرتين:

مرة بدون احتساب يوم النهاية

ومرة مع احتساب يوم النهاية

مثال:

Start Date: 1/1/2022

End Date: 5/1/2022

Period Length is: 4

Period Length (Including End Date) is: 5

تفسير:

يتم حساب الفرق بين اليوم الأول واليوم الخامس كالتالي:

1→2، 2→3، 3→4، 4→5 4 أيام بدون احتساب يوم النهاية، لكن إذا شملنا يوم النهاية تصبح 5 أيام

ملاحظة:

البرنامج لا يتحقق من صحة ترتيب التواريخ، لذا يجب أن يكون تاريخ البداية قبل تاريخ النهاية يدويًا

✨ خلاصة:

- ✓ البرنامج يحسب طول الفترة الزمنية بدقة حتى عبر الأشهر والسنوات
- ✓ يدعم خيار حساب المدة مع أو بدون يوم النهاية
- ✓ مفيد لتطبيقات إدارة الوقت أو العطل أو الفواتير

Problem #60: Is Date Within Period

التعريف:

sDate Struct: يمثل تاريخًا واحدًا يحتوي على اليوم، الشهر، والسنة

sPeriod Struct: يمثل فترة زمنية محددة ببداية وانتهاء:

StartDate: تاريخ البداية

EndDate: تاريخ النهاية

enDateCompare Enum: لتحديد العلاقة بين تاريخين:

Before = -1: التاريخ الأول قبل الثاني

Equal = 0: التاريخان متساويان

After = 1: التاريخ الأول بعد الثاني

الشرح:

هذا البرنامج يقرأ فترة زمنية (تاريخ بداية ونهاية) ثم يقرأ تاريخًا آخر، ويقوم بالتحقق مما إذا كان هذا التاريخ يقع داخل الفترة أم لا (بما في ذلك يوم البداية والنهاية)

الإجراءات:

1 قراءة تاريخ البداية للفترة

2 قراءة تاريخ النهاية للفترة

3 قراءة تاريخ للتحقق منه

4 مقارنة التاريخ المُدخل مع بداية ونهاية الفترة

5 تحديد ما إذا كان يقع ضمن الفترة

6 طباعة النتيجة

الدوال الأساسية:

IsDate1BeforeDate2(date1, date2): تتحقق إذا كان التاريخ الأول قبل الثاني

IsDate1EqualDate2(date1, date2): تتحقق إذا كان التاريخان متساويين

IsDate1AfterDate2(date1, date2): تتحقق إذا كان التاريخ الأول بعد الثاني

CompareDates(date1, date2): تُرجع قيمة من نوع **enDateCompare** لتحديد العلاقة بين التاريخين

isDateInPeriod(date, period): تُرجع **true** إذا كان التاريخ ضمن الفترة، أو **false** إذا لم يكن كذلك

مثال:

Enter Period:

Start Date: 1/1/2022

End Date: 10/1/2022

Date to check: 5/1/2022

Yes, Date is within period.

تفسير:

البرنامج يقارن التاريخ المُدخل مع بداية ونهاية الفترة:

إذا لم يكن قبلهما أو بعدهما، فهو ضمن الفترة

أي أن العلاقة المنطقية المستخدمة:

NOT (قبل البداية OR بعد النهاية)

ملاحظة:

يجب أن تكون تواريخ البداية والنهاية مرتبة زمنيًا بشكل صحيح (أي لا يمكن أن تكون البداية بعد النهاية) 

خلاصة:

- ✓ البرنامج يتحقق بدقة مما إذا كان تاريخ معين يقع داخل فترة زمنية محددة
- ✓ يمكن استخدامه في تطبيقات مثل إدارة الإجازات أو الحجوزات أو التحقق من صلاحية تواريخ معينة

Problem #61: Count Overlap Days

التعريف:

sDate Struct: يمثل تاريخًا واحدًا يحتوي على اليوم، الشهر، والسنة

sPeriod Struct: يمثل فترة زمنية تحتوي على:

StartDate: تاريخ البداية

EndDate: تاريخ النهاية

enDateCompare Enum

لتحديد العلاقة بين تاريخين:

Before = -1: التاريخ الأول قبل الثاني

Equal = 0: التاريخان متساويان

After = 1: التاريخ الأول بعد الثاني

الشرح:

هذا البرنامج يقرأ فترتين زمنيتين (كل فترة تحتوي على تاريخ بداية وتاريخ نهاية) ثم يحسب عدد الأيام المشتركة بين الفترتين (الأيام المتداخلة)

الإجراءات:

1 قراءة الفترة الأولى (بداية ونهاية)

2 قراءة الفترة الثانية (بداية ونهاية)

3 التحقق هل هناك تداخل بين الفترتين

4 إذا كان هناك تداخل، يتم حساب الأيام المشتركة بينهما

5 طباعة عدد الأيام المتداخلة

الدوال الأساسية:

IsDate1BeforeDate2(date1, date2): تتحقق إذا كان التاريخ الأول قبل الثاني

IsDate1EqualDate2(date1, date2): تتحقق إذا كان التاريخان متساويين

IsDate1AfterDate2(date1, date2): تتحقق إذا كان التاريخ الأول بعد الثاني

IsLeapYear(year): تتحقق إذا كانت السنة كبيسة (29 يومًا في فبراير)

NumberOfDaysInAMonth(month, year): تُعيد عدد الأيام في الشهر المحدد

IncreaseDateByOneDay(date): تُزيد التاريخ يومًا واحدًا مع التعامل مع تغيير الشهر والسنة تلقائيًا

GetDifferenceInDays(date1, date2, includeEndDay): تُعيد الفرق بعدد الأيام بين تاريخين

IsOverlapPeriods(period1, period2): تتحقق إذا كان هناك تداخل بين فترتين

IsDateInPeriod(date, period): تتحقق إذا كان تاريخ معين يقع داخل فترة

CountOverlapDays(period1, period2): تحسب عدد الأيام المتداخلة بين فترتين

مثال:

Enter Period 1:

Start Date: 1/1/2022

End Date: 10/1/2022

Enter Period 2:

Start Date: 5/1/2022

End Date: 30/12/2050

Overlap Days Count Is: 5

🧐 تفسير:

- البرنامج يمرّ على كل يوم في الفترة الأقصر، ويتحقق إن كان هذا اليوم ضمن الفترة الثانية
- كل يوم متداخل يزيد عداد الأيام المشتركة

⚖️ مقارنة:

الحالة	النتيجة
لا يوجد تداخل (إحدى الفترات تنتهي قبل بداية الأخرى)	0 أيام
يوجد تداخل جزئي	يحسب الأيام المشتركة فقط
إحدى الفترتين داخل الأخرى	يحسب كامل الفترة الصغيرة

📌 ملاحظة:

- إذا كانت الفترات مقلوبة (تاريخ البداية بعد النهاية)، النتائج ستكون خاطئة. يجب التأكد من ترتيب التواريخ أولاً

✨ خلاصة:

- ✓ البرنامج يحسب عدد الأيام المشتركة بين فترتين زمنيّتين بدقة من خلال مقارنة يومية منظمة
- ✓ يُستخدم في التطبيقات التي تتعلق بالإجازات، الحجوزات، أو تحليل الفترات الزمنية

Problem #62: Validate Date

التعريف:

sDate Struct: يمثل التاريخ ويتكون من:

Day اليوم

Month الشهر

Year السنة

(IsLeapYear(year): تحدد ما إذا كانت السنة كبيسة أم لا

السنة كبيسة إذا كانت:

تقبل القسمة على 400، أو تقبل القسمة على 4 ولا تقبل على 100

(NumberOfDaysInAMonth(month, year: تُرجع عدد الأيام في الشهر، مع مراعاة السنة الكبيسة لشهر فبراير

(IsValidDate(date: تتحقق من أن اليوم والشهر يقعان ضمن النطاق الصحيح، وأن اليوم لا يتجاوز عدد الأيام المسموح بها في ذلك الشهر

الشرح:

هذا البرنامج يتحقق من صحة تاريخ مُدخل من قبل المستخدم (اليوم، الشهر، السنة) ويتأكد مما إذا كان التاريخ صالحًا أم لا من حيث عدد الأيام في الشهر، وصحة الشهر، والسنة الكبيسة

الإجراءات:

- 1 المستخدم يدخل اليوم، الشهر، والسنة
- 2 يتم حساب عدد الأيام المسموح بها للشهر المعطى
- 3 يتم التحقق من أن القيم المدخلة منطقية وصحيحة
- 4 تُعرض النتيجة إن كان التاريخ صحيحًا أو غير صالح

مثال:

Day = 35, Month = 1, Year = 2022
No, Date is NOT valid date

Day = 29, Month = 2, Year = 2022
No, Date is NOT valid date

Day = 29, Month = 2, Year = 2000
Yes, Date is a valid date.

Day = 31, Month = 4, Year = 2022
No, Date is NOT valid date

ملاحظة:

إذا كان الشهر أو اليوم خارج النطاق (مثل شهر 13 أو يوم 0) فسيُعتبر التاريخ غير صالح مباشرة

🧐 تفسير:

- البرنامج يعتمد على مقارنة اليوم المدخل بعدد الأيام الفعلي للشهر
- يتم استدعاء دالة `IsLeapYear()` لتحديد إذا كان فبراير يحتوي على 28 أو 29 يومًا

✨ خلاصة:

✓ البرنامج يتحقق من صحة التاريخ بشكل شامل باستخدام الشروط التالية:

- اليوم بين 1 و 31
- الشهر بين 1 و 12
- عدد الأيام لا يتجاوز ما يسمح به الشهر
- يتعامل مع السنة الكبيسة بشكل صحيح

Problem #63 – #64: Read/Print Date String

التعريف:

برنامج لقراءة تاريخ بصيغة نصية (String) مثل 31/3/2022، وتحويله إلى هيكل بيانات (Struct) يحتوي على اليوم، الشهر، والسنة، ثم طباعتها بشكل منفصل، وبعدها إعادة تحويل الهيكل إلى نص مرة أخرى

الشرح:

الهدف من البرنامج هو التعامل مع التاريخ بشكل مزدوج (كنص وكهيكل بيانات)
يستخدم البرنامج عدة دوال لتقسيم النص، التحويل بين النص والبنية، والطباعة

الإجراءات:

1 تعريف struct باسم sDate

يحتوي على ثلاث قيم:

short Day

short Month

short Year

2 دالة SplitString

التعريف: تقسم النص إلى أجزاء حسب محدد (delimiter)

تستخدم هنا للفصل بين اليوم، الشهر، والسنة عند وجود /

ملاحظة: تُعيد دالة SplitString متجهًا (vector<string>) يحتوي على الأجزاء المفصلة

3 دالة StringToDate

التعريف: تحول النص إلى متغير من نوع sDate

تعتمد على SplitString لاستخراج الأجزاء

تستخدم stoi لتحويل النصوص إلى أرقام

مثال: Day=31, Month=3, Year=2022 → "31/3/2022"

4 دالة DateToString

التعريف: تعيد تحويل هيكل التاريخ إلى نص مفصول بعلامة /

مثال: "31/3/2022" → (2022, 3, 31)

5 دالة ReadStringDate

التعريف: لقراءة النص من المستخدم

تستخدم (getline(cin >> ws, dateString)) لتفادي المشكلات الناتجة عن المسافات

6 الدالة الرئيسية main()

تقرأ التاريخ النصي من المستخدم

تحول النص إلى تاريخ باستخدام StringToDate

تطبع اليوم، الشهر، السنة كل على حدة

ثم تطبع النص المعاد تحويله باستخدام DateToString

مثال:

Please Enter Date dd/mm/yyyy? 31/3/2022

Day: 31

Month: 3

Year: 2022

You Entered: 31/3/2022.

✨ خلاصة:

- ✓ البرنامج يوضح التعامل مع التواريخ بين النصوص والهياكل
- ✓ يعتمد على تقسيم النصوص (`SplitString`) والتحويل (`stoi` و `to_string`)
- ✓ يسهل لاحقاً مقارنة التواريخ أو تنفيذ عمليات زمنية عليها

Problem #65: FormatDate

التعريف:

برنامج يقرأ تاريخًا من المستخدم بصيغة نصية مثل **31/12/2022**، ثم يعرضه بعدة تنسيقات مختلفة (Formats) حسب الصيغة المطلوبة مثل:

31/12/2022

2022/31/12

12-31-2022

Day: 31, Month: 12, Year: 2022

الشرح:

الهدف من البرنامج هو تخصيص طريقة عرض التاريخ باستخدام دالة **FormateDate** التي تعتمد على استبدال الرموز داخل النص (**dd, mm, yyyy**) بقيم اليوم، الشهر، والسنة من الهيكل **sDate**

الإجراءات:

1 تعريف struct sDate

يحتوي على ثلاثة عناصر:

Day

Month

Year

2 دالة SplitString

التعريف: تقسم النص إلى أجزاء بناءً على محدد معين (مثل "/")

تُستخدم لاستخراج اليوم، الشهر، السنة من النص المدخل

3 دالة StringToDate

التعريف: تحول النص **dd/mm/yyyy** إلى متغير من نوع **sDate**

تعتمد على دالة **SplitString** ثم تستخدم **stoi** لتحويل النصوص إلى أرقام صحيحة

4 دالة DateToString

التعريف: تحول هيكل التاريخ إلى نص بصيغة **"dd/mm/yyyy"**

5 دالة ReplaceWordInString

التعريف: تستبدل كل ظهور لكلمة داخل نص بكلمة أخرى

تُستخدم هنا لتبديل **dd, mm, yyyy** بالقيم الحقيقية للتاريخ

6 دالة FormateDate

التعريف: تنشئ التاريخ بصيغة محددة من المستخدم

تأخذ معاملين:

sDate date (التاريخ نفسه)

string dateFormat (النمط المطلوب مثل **"mm-dd-yyyy"**)

ملاحظة: إذا لم يحدد المستخدم تنسيقًا، يتم استخدام **"dd/mm/yyyy"** كتتنسيق افتراضي

7 الدالة الرئيسية (main)

تطلب من المستخدم إدخال تاريخ

تحوله إلى **sDate**

تطبع عدة صيغ للتاريخ باستخدام **FormateDate**

مثال:

Please Enter Date dd/mm/yyyy? 31/12/2022

31/12/2022

2022/31/12

12/31/2022

12-31-2022

31-12-2022

Day:31, Month:12, Year:2022

✨ خلاصة:

- ✓ البرنامج يوضح مفهوم تنسيق المخرجات (Formatting)
- ✓ يعتمد على التحويل بين النص والهيكل
- ✓ يمكن استخدامه لعرض التاريخ بصيغ مختلفة حسب متطلبات النظام أو المستخدم